

Python & FastAPI Backend Development

Interview-Oriented Learning Documentation

Submitted By: Parth

Topic: Python Backend Development using FastAPI

Purpose: To document the concepts, examples, and code learned for interview and backend development preparation

This documentation summarizes the topics I studied from backend basics to FastAPI core concepts, including request-response flow, REST APIs, Python fundamentals, routing, Pydantic schemas, CRUD APIs, authentication basics, middleware, CORS, and backend architecture.

Table of Contents

1. Backend Basics
2. Python Basics for Backend Development
3. FastAPI Core Concepts
4. CRUD API Mini Project
5. Authentication Basics
6. Middleware, CORS, and Backend Architecture
7. Important Interview Questions and Answers
8. Final Summary

1. Backend Basics

1.1 Frontend vs Backend

A modern application mainly has three parts: frontend, backend, and database. The frontend is the visible part of the application, while the backend handles logic, security, authentication, APIs, and database communication.

Part	Meaning	Example Technologies
Frontend	User interface that users can see and interact with	HTML, CSS, JavaScript, React, Next.js
Backend	Server-side logic, APIs, authentication, and database communication	Python FastAPI, Django, Flask, Node.js
Database	Stores application data	MySQL, PostgreSQL, MongoDB, SQLite

Example: In a login system, the email/password form is frontend, while checking credentials from the database is backend.

1.2 Client, Server, Request, Response

- Client: The application that sends a request, such as browser, mobile app, React frontend, or Postman.
- Server: The application that receives the request, processes it, and sends a response.
- Request: Message sent by the client to the server.
- Response: Result sent by the server back to the client.

```
Client -> Request -> Server
Client <- Response <- Server
```

1.3 API

API stands for Application Programming Interface. It works as a bridge between frontend and backend. The frontend does not directly communicate with the database. It sends requests to APIs, and the backend handles validation, logic, database operations, and response.

```
Frontend -> API -> Backend -> Database
Frontend <- API Response <- Backend
```

1.4 REST API

REST API is a standard style for designing APIs where resources are accessed using HTTP methods.

HTTP Method	Purpose	Example
GET	Fetch/read data	GET /users
POST	Create new data	POST /users
PUT	Full update	PUT /users/1
PATCH	Partial update	PATCH /users/1
DELETE	Delete data	DELETE /users/1

1.5 HTTP Status Codes

Status Code	Meaning	Common Use
200	OK	Request successful
201	Created	New resource created

400	Bad Request	Invalid input or business rule failure
401	Unauthorized	Token missing or invalid
403	Forbidden	User logged in but no permission
404	Not Found	Resource does not exist
422	Validation Error	Pydantic validation failed
500	Server Error	Unexpected backend/server issue

2. Python Basics for Backend Development

2.1 Variables and Data Types

A variable stores data. Python is dynamically typed, so we do not need to declare the type manually. Python decides the type at runtime.

```
name = "Parth"      # str
age = 21            # int
marks = 85.5        # float
is_active = True    # bool
```

Data Type	Meaning	Example
int	Whole number	21
float	Decimal number	85.5
str	Text	"Parth"
bool	True/False	True
list	Ordered mutable collection	[1, 2, 3]
tuple	Ordered immutable collection	(1, 2, 3)
set	Unique unordered collection	{1, 2, 3}
dict	Key-value pair	{"name": "Parth"}
None	No value	None

2.2 List, Tuple, Set, Dictionary

- List: Ordered, changeable, allows duplicates. Used for multiple records like users/products.
- Tuple: Ordered, unchangeable. Used for fixed data like allowed roles.
- Set: Unordered and stores only unique values. Useful for duplicate removal.
- Dictionary: Key-value pair structure. Very important because JSON responses are similar to Python dictionaries.

```
users = [
    {"id": 1, "name": "Parth", "is_active": True},
    {"id": 2, "name": "Aman", "is_active": False}
]

for user in users:
    if user["is_active"]:
        print(user["name"])
```

2.3 Conditions

Conditions are used for decision making. In backend, conditions are used for login validation, role checking, user existence check, stock availability, and error handling.

```
email_correct = True
password_correct = True
```

```
if email_correct and password_correct:
    print("Login successful")
else:
    print("Invalid login")
```

2.4 Loops

Loops are used to repeat a block of code. In backend development, loops are commonly used to process lists of users, products, orders, or records.

```
users = [
    {"id": 1, "name": "Parth"},
    {"id": 2, "name": "Aman"}
]

search_id = 2
for user in users:
    if user["id"] == search_id:
        print(user)
        break
```

2.5 Functions

A function is a reusable block of code. Functions help avoid repetition and make backend logic modular.

```
def check_password(entered_password, stored_password):
    if entered_password == stored_password:
        return True
    return False

result = check_password("12345", "12345")
print(result)
```

2.6 Exception Handling

Exception handling prevents the program from crashing when runtime errors occur. In backend APIs, proper error responses should be returned instead of exposing raw errors.

```
try:
    number = int(input("Enter number: "))
    result = 10 / number
    print(result)
except ZeroDivisionError:
    print("Cannot divide by zero")
except ValueError:
    print("Please enter a valid number")
finally:
    print("Program completed")
```

2.7 Modules, Imports, and OOP

A module is a Python file containing reusable code. Imports help reuse functions, classes, and variables from other files or libraries. OOP organizes code using classes and objects.

```
from math import sqrt
print(sqrt(16))

class User:
```

```
def __init__(self, name, email):
    self.name = name
    self.email = email

user1 = User("Parth", "parth@gmail.com")
print(user1.name)
```

OOP Concept	Meaning
Encapsulation	Binding data and methods together inside a class
Inheritance	One class can reuse properties and methods of another class
Polymorphism	Same method name can behave differently for different objects
Abstraction	Hiding internal implementation and showing only important features

3. FastAPI Core Concepts

3.1 FastAPI Setup and First API

FastAPI is a modern Python framework used to build REST APIs. Uvicorn is an ASGI server used to run FastAPI applications.

```
pip install fastapi uvicorn
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Hello FastAPI"}

uvicorn main:app --reload
```

- Swagger documentation is available at: <http://127.0.0.1:8000/docs>
- FastAPI automatically converts Python dictionaries into JSON responses.

3.2 Routing

Routing means mapping a URL path and HTTP method to a Python function.

```
@app.get("/users")
def get_users():
    return {"message": "All users"}

@app.post("/users")
def create_user():
    return {"message": "User created"}
```

3.3 Path Parameters and Query Parameters

```
@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id}
```

```
@app.get("/products")
def get_products(category: str, limit: int = 10):
    return {"category": category, "limit": limit}
```

Parameter Type	Purpose	Example
Path Parameter	Identify a specific resource	/users/10
Query Parameter	Filtering, searching, sorting, pagination	/products?category=mobile&limit=5

3.4 Request Body and Pydantic Schemas

Request body is data sent by the client to the backend, usually in POST, PUT, or PATCH requests. Pydantic schemas validate that data.

```
from pydantic import BaseModel

class UserCreate(BaseModel):
    name: str
    email: str
    password: str

@app.post("/users")
def create_user(user: UserCreate):
    return {"name": user.name, "email": user.email}
```

- If request data does not match the schema, FastAPI returns 422 validation error.
- Separate schemas should be used for input and output, such as UserCreate and UserResponse.
- Passwords should not be returned in API responses.

3.5 response_model

```
class UserResponse(BaseModel):
    id: int
    name: str
    email: str

@app.post("/users", response_model=UserResponse)
def create_user(user: UserCreate):
    return {
        "id": 1,
        "name": user.name,
        "email": user.email,
        "password": user.password
    }
```

Even if password is returned internally, response_model filters it out and sends only id, name, and email.

4. CRUD API Mini Project

CRUD means Create, Read, Update, and Delete. I created a basic User CRUD API using FastAPI, Pydantic schemas, status codes, and HTTPException.

```
from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel
from typing import Optional
```

```

app = FastAPI()

class UserCreate(BaseModel):
    name: str
    email: str
    password: str

class UserUpdate(BaseModel):
    name: Optional[str] = None
    email: Optional[str] = None

class UserResponse(BaseModel):
    id: int
    name: str
    email: str

users = []
next_id = 1

@app.get("/users", response_model=list[UserResponse])
def get_users():
    return users

@app.get("/users/{user_id}", response_model=UserResponse)
def get_user(user_id: int):
    for user in users:
        if user["id"] == user_id:
            return user
    raise HTTPException(status_code=404, detail="User not found")

@app.post("/users", response_model=UserResponse, status_code=status.HTTP_201_CREATED)
def create_user(user: UserCreate):
    global next_id
    for existing_user in users:
        if existing_user["email"] == user.email:
            raise HTTPException(status_code=400, detail="Email already registered")
    new_user = {"id": next_id, "name": user.name, "email": user.email, "password": user.password}
    users.append(new_user)
    next_id += 1
    return new_user

@app.patch("/users/{user_id}", response_model=UserResponse)
def update_user(user_id: int, user_update: UserUpdate):
    for user in users:
        if user["id"] == user_id:
            if user_update.name is not None:
                user["name"] = user_update.name
            if user_update.email is not None:
                user["email"] = user_update.email
            return user
    raise HTTPException(status_code=404, detail="User not found")

@app.delete("/users/{user_id}")
def delete_user(user_id: int):
    for user in users:
        if user["id"] == user_id:
            users.remove(user)

```



```

    return {"message": "User deleted successfully"}
    raise HTTPException(status_code=404, detail="User not found")

```

Endpoint	Method	Purpose
/users	GET	Fetch all users
/users/{user_id}	GET	Fetch one user
/users	POST	Create a new user
/users/{user_id}	PATCH	Update selected fields
/users/{user_id}	DELETE	Delete user

5. Authentication Basics

Concept	Meaning
Authentication	Verifies who the user is
Authorization	Checks what the authenticated user is allowed to do
JWT	Token used for login-based API access
Bearer Token	Token format sent in Authorization header
Password Hashing	Securely stores passwords in irreversible hash form
Protected Route	API that requires a valid token

Login flow: user enters email and password, backend verifies credentials, generates JWT token, frontend stores token, and sends it in the Authorization header for protected APIs.

```

Authorization: Bearer <token>

from fastapi import FastAPI, HTTPException, Header

app = FastAPI()

@app.get("/profile")
def profile(authorization: str = Header(None)):
    if authorization != "Bearer dummy-token":
        raise HTTPException(status_code=401, detail="Invalid or missing token")
    return {"name": "Parth", "email": "parth@gmail.com"}

```

- 401 Unauthorized means token is missing or invalid.
- 403 Forbidden means user is logged in but does not have permission.
- Plain passwords should never be stored in real projects; they should be hashed using bcrypt/passlib.

6. Middleware, CORS, and Backend Architecture

6.1 Middleware

Middleware is code that runs before and after request processing. It is used for logging, CORS, authentication checks, security headers, and error handling.

```

@app.middleware("http")
async def log_requests(request, call_next):
    print("Request received:", request.url)
    response = await call_next(request)

```

```
print("Response sent")
return response
```

6.2 CORS

CORS stands for Cross-Origin Resource Sharing. It controls whether a frontend from one origin can access backend APIs from another origin.

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

6.3 Clean Backend Architecture

```
app/
├── main.py
├── database.py
├── config.py
├── routes/
│   └── user_routes.py
├── schemas/
│   └── user_schema.py
├── models/
│   └── user_model.py
├── services/
│   └── user_service.py
├── auth/
│   └── jwt_handler.py
```

Layer/File	Work
main.py	Application starting point
routes/	API endpoints
schemas/	Pydantic request/response validation
models/	Database table structure
services/	Business logic
database.py	Database connection/session
auth/	JWT, login, password hashing
config.py	Environment variables and settings

```
Request -> Route -> Schema Validation -> Service Logic -> Database/Model -> Response
```

7. Important Interview Questions and Answers

Q. What is FastAPI?

Answer: FastAPI is a modern Python framework used to build REST APIs with automatic validation, documentation, type hints, async support, and high performance.

Q. What is an API?

Answer: API is a bridge between frontend and backend. It allows clients to send requests and receive responses from the server.

Q. What is REST API?

Answer: REST API is an API design style where resources are accessed using HTTP methods like GET, POST, PUT, PATCH, and DELETE.

Q. What is Pydantic?

Answer: Pydantic is used for data validation and serialization in FastAPI using Python type hints.

Q. What is request body?

Answer: Request body is the data sent by the client to the server, usually in POST, PUT, or PATCH requests.

Q. What is response_model?

Answer: response_model defines and validates the response structure and can filter sensitive fields like passwords.

Q. What is HTTPException?

Answer: HTTPException is used to send proper error responses with status codes and messages.

Q. Authentication vs Authorization?

Answer: Authentication verifies identity. Authorization checks permissions.

Q. What is JWT?

Answer: JWT stands for JSON Web Token. It is used for token-based authentication after login.

Q. What is CORS?

Answer: CORS is a browser security mechanism that controls whether a frontend from one origin can access APIs from another origin.

Q. Schema vs Model?

Answer: Schema is used for request/response validation, while model represents the database table structure.

Q. Why clean architecture?

Answer: Clean architecture separates routes, schemas, models, services, database, and auth logic, making the project scalable and maintainable.

8. Final Summary

Through this learning documentation, I studied Python backend development basics and FastAPI core concepts. I learned how frontend and backend communicate through APIs, how REST APIs are designed, how HTTP methods and status codes work, and how Python concepts such as variables, data types, conditions, loops, functions, exception handling, modules, and OOP are used in backend development.

I also learned FastAPI routing, request body handling, Pydantic schemas, response models, CRUD APIs, HTTPException, authentication basics, JWT token flow, password hashing, protected routes, middleware, CORS, and clean backend architecture. This knowledge helps me understand how production-level backend APIs are designed and implemented.